ETS

PEMROGRAMAN IMCLAB BERBASIS BLYNK



Dosen Pengampu:

Dr. Basuki Rahmat, S.Si. MT

Disusun oleh:

Arif Nur Cahyo (23081010192)

ROBOTIKA A – 081

PROGRAM STUDI INFORMATIKA

FAKULTAS ILMU KOMPUTER

UNIVERSITAS PEMBANGUNAN NASIONAL " VETERAN"

JAWA TIMUR

2026

DAFTAR ISI

DAFTAR ISI i
DAFTAR GAMBAR ii
BAB I KONFIGURASI DAN KODE PROGRAM 1
1.1 Konfigurasi Template dan Datastream Blynk 1
1.2 Daftar Perintah Serial 2
1.3 Kode Program 2
BAB II HASIL DAN PEMBAHASAN 7
2.1 Proses Upload Program ke ESP32 7
2.2 Koneksi Perangkat ke Server Blynk 7
2.3 Pengujian Tombol Switch (Motor ON) 8
2.4 Menaikkan Kecepatan melalui Slider 9
2.5 Menurunkan Kecepatan melalui Slider 10
2.6 Mematikan Motor melalui Switch 11
2.7 Pembacaan RPM dan Keterbatasan Skala Gauge 11
2.8 Rangkuman Hasil Pengujian 12
BAB III PENUTUP 13
3.1 Kesimpulan 13
3.2 Saran 13

DAFTAR GAMBAR

Gambar 1. Proses upload program Blynk ke board ESP32 7
Gambar 2. Dashboard Blynk saat perangkat berstatus Online.....8
Gambar 3. Pengujian langsung dashboard Blynk dengan KIT iMCLab 9
Gambar 4. Kecepatan dinaikkan melalui slider menjadi PWM 207.....10
Gambar 5. Kecepatan diturunkan melalui slider menjadi PWM 195 11
Gambar 6. Kondisi dashboard setelah Switch dimatikan.....11

BAB I KONFIGURASI DAN KODE PROGRAM

Pada praktikum ini pengendalian motor DC pada KIT iMCLab dilakukan melalui platform IoT Blynk. Berbeda dengan dua studi sebelumnya yang mengandalkan Serial Monitor maupun program Python pada Jupyter Notebook, pengendalian kali ini berlangsung melalui jaringan internet sehingga motor dapat dikendalikan dari dashboard web Blynk tanpa perlu terhubung langsung ke komputer melalui kabel data.

Board ESP32 terhubung ke jaringan WiFi, kemudian menyambung ke server Blynk Cloud menggunakan Template ID dan Auth Token. Selain mengendalikan motor, program juga membaca sensor encoder pada pin 13 untuk menghitung putaran per menit (RPM) dan mengirimkan hasilnya ke dashboard setiap satu detik melalui BlynkTimer.

1.1 Konfigurasi Template dan Datastream Blynk

Program memerlukan tiga identitas yang harus sama persis dengan yang terdaftar pada Blynk Console, yaitu Template ID, Template Name, dan Auth Token milik device. Pertukaran data antara ESP32 dan dashboard dilakukan melalui empat virtual pin berikut.

Pin	Nama	Arah	Keterangan
V0	RPM	ESP32 → Blynk	Hasil pembacaan sensor encoder, dikirim tiap 1 detik
V1	PWM	ESP32 → Blynk	Nilai duty cycle yang sedang aktif (0–255)
V2	Slider PWM	dua arah	Masukan pengguna untuk mengatur duty cycle
V3	Switch	dua arah	Tombol ON/OFF motor

Fungsi BLYNK_WRITE(V2) menangani perubahan slider, sedangkan BLYNK_WRITE(V3) menangani tombol ON/OFF. Sebaliknya, fungsi kirimKeBlynk() dan applyMotor() menuliskan nilai terbaru kembali ke dashboard sehingga tampilan selalu sinkron dengan kondisi perangkat. Fungsi BLYNK_CONNECTED() memanggil Blynk.syncVirtual(V2) dan

Blynk.syncVirtual(V3) agar status terakhir dipulihkan setiap kali perangkat terhubung ulang.

1.2 Daftar Perintah Serial

Selain melalui dashboard, program tetap menerima perintah dari Serial Monitor sebagai sarana pengujian cadangan.

No	Perintah	Fungsi
1	‘1’	Menyalakan motor dengan duty cycle terakhir atau nilai default 200
2	‘0’	Mematikan motor (duty cycle menjadi 0)
3	‘+’	Menaikkan duty cycle sebesar 10 satuan
4	‘-’	Menurunkan duty cycle sebesar 10 satuan
5	‘PWM:xxx’	Menetapkan duty cycle secara langsung pada rentang 0–255
6	‘R’	Menampilkan nilai RPM dan PWM terkini

1.3 Kode Program

Berikut adalah kode program lengkap imclab_Blynk_kelas.ino yang digunakan pada praktikum ini.

```
/*************************************************************
 * iMCLab + ESP32 + Blynk IoT
 * RPM Monitoring dan Kontrol PWM
 *************************************************************/

#define BLYNK_TEMPLATE_ID "TMPL6rGehYQam"
#define BLYNK_TEMPLATE_NAME "robot"
#define BLYNK_AUTH_TOKEN "DDFLFzKPinPkVqKgA_36G48y0Al5ibPc"

#include <WiFi.h>
#include <BlynkSimpleEsp32.h>

//char ssid[] = "HOTSPOT@UPNJATIM.AC.ID";
//char pass[] = "belanegara";

//Konfigurasi Wifi
char auth[] = BLYNK_AUTH_TOKEN;
char ssid[] = "HOTSPOT@UPNJATIM.AC.ID";
char pass[] = "belanegara";


// ============================================================
// Motor dan LED
// ============================================================

int ledPin = 2;

int motor1Pin1 = 27;
int motor1Pin2 = 26;
int enable1Pin = 12;
```

```
// ============================================================
// Sensor RPM
// ============================================================

const byte pin_rpm = 13;
const int HOLES = 2;

volatile unsigned int rev = 0;
volatile unsigned long lastPulseTime = 0;

const unsigned long DEBOUNCE_US = 500;

// ============================================================
// PWM
// ============================================================

const int freq = 30000;
const int pwmChannel = 0;
const int resolution = 8;

int dutyCycle = 200;

const int DUTY_STEP = 10;
const int DUTY_MIN = 0;
const int DUTY_MAX = 255;

bool motorEnable = false;

// ============================================================
// RPM
// ============================================================

float rpm = 0.0;
unsigned long old_time = 0;

// ============================================================

String inputString = "";

BlynkTimer timer;

// ============================================================
// ISR Encoder
// ============================================================

void IRAM_ATTR isr()
{
  unsigned long now = micros();

  if (now - lastPulseTime > DEBOUNCE_US)
  {
    rev++;
    lastPulseTime = now;
  }
}

// ============================================================
// Sync setelah reconnect Blynk
// ============================================================

BLYNK_CONNECTED()
{
  Blynk.syncVirtual(V2);
  Blynk.syncVirtual(V3);
}

// ============================================================
// Hitung RPM
// ============================================================

void hitungRPM()
{
  unsigned long cur_time = millis();

  if (cur_time - old_time >= 1000)
  {
    detachInterrupt(digitalPinToInterrupt(pin_rpm));
```

```
    float rot_per_sec = (float)rev / (float)HOLES;

    rpm = rot_per_sec * 60.0;

    rev = 0;

    attachInterrupt(
      digitalPinToInterrupt(pin_rpm),
      isr,
      FALLING);

    old_time = millis();
  }
}

// ============================================================
// Terapkan PWM ke Motor
// ============================================================

void applyMotor()
{
  digitalWrite(motor1Pin1, HIGH);
  digitalWrite(motor1Pin2, LOW);

  ledcWrite(pwmChannel, dutyCycle);

  digitalWrite(ledPin, dutyCycle > 0);

  // Sinkronisasi ke Blynk
  Blynk.virtualWrite(V1, dutyCycle);
  Blynk.virtualWrite(V2, dutyCycle);
  Blynk.virtualWrite(V3, dutyCycle > 0);
}

// ============================================================
// Status Serial
// ============================================================

void tampilkanStatus(const char* aksi)
{
  Serial.print(aksi);
  Serial.print(" | PWM=");
  Serial.print(dutyCycle);
  Serial.print(" | RPM=");
  Serial.println(rpm, 1);
}

// ============================================================
// Kirim Data ke Blynk
// ============================================================

void kirimKeBlynk()
{
  Blynk.virtualWrite(V0, rpm);
  Blynk.virtualWrite(V1, dutyCycle);
  Blynk.virtualWrite(V2, dutyCycle);
  Blynk.virtualWrite(V3, dutyCycle > 0);
}

// ============================================================
// Slider PWM dari Blynk (V2)
// ============================================================

BLYNK_WRITE(V2)
{
  dutyCycle = constrain(
    param.asInt(),
    DUTY_MIN,
    DUTY_MAX);

  motorEnable = (dutyCycle > 0);

  applyMotor();

  Serial.print("PWM dari Blynk = ");
  Serial.println(dutyCycle);
```

```
}

// ============================================================
// Tombol ON/OFF dari Blynk (V3)
// ============================================================

BLYNK_WRITE(V3)
{
  motorEnable = param.asInt();

  if (motorEnable)
  {
    if (dutyCycle == 0)
      dutyCycle = 200;
  }
  else
  {
    dutyCycle = 0;
  }

  applyMotor();

  Serial.print("Motor ");
  Serial.println(motorEnable ? "ON" : "OFF");
}

// ============================================================
// Setup
// ============================================================

void setup()
{
  Serial.begin(115200);

  pinMode(ledPin, OUTPUT);

  pinMode(motor1Pin1, OUTPUT);
  pinMode(motor1Pin2, OUTPUT);
  pinMode(enable1Pin, OUTPUT);

  pinMode(pin_rpm, INPUT_PULLUP);

  attachInterrupt(
    digitalPinToInterrupt(pin_rpm),
    isr,
    FALLING);

  ledcSetup(
    pwmChannel,
    freq,
    resolution);

  ledcAttachPin(
    enable1Pin,
    pwmChannel);

  digitalWrite(motor1Pin1, HIGH);
  digitalWrite(motor1Pin2, LOW);

  ledcWrite(pwmChannel, 0);

  old_time = millis();

//  Blynk.begin(
//    BLYNK_AUTH_TOKEN,
//    ssid,
//    pass);

  // Koneksi ke Blynk
  Blynk.begin(auth, ssid, pass);

  timer.setInterval(
    1000L,
    kirimKeBlynk);

  Serial.println("================================");
  Serial.println("iMCLab + Blynk Ready");
```

```
  Serial.println("================================");
  Serial.println("1 = ON");
  Serial.println("0 = OFF");
  Serial.println("+ = PWM naik");
  Serial.println("- = PWM turun");
  Serial.println("PWM:200");
  Serial.println("R = lihat RPM");
}

// ============================================================
// Loop
// ============================================================

void loop()
{
  Blynk.run();
  timer.run();

  hitungRPM();

  while (Serial.available() > 0)
  {
    char c = Serial.read();

    if (c == '\n' || c == '\r')
    {
      inputString.trim();

      if (inputString.startsWith("PWM:"))
      {
        int val = inputString.substring(4).toInt();

        val = constrain(
          val,
          DUTY_MIN,
          DUTY_MAX);

        dutyCycle = val;
        motorEnable = (dutyCycle > 0);

        applyMotor();

        tampilkanStatus("Set PWM");
      }

      else if (inputString.length() > 0)
      {
        char cmd = inputString.charAt(0);

        if (cmd == '1')
        {
          if (dutyCycle == 0)
            dutyCycle = 200;

          motorEnable = true;

          applyMotor();

          tampilkanStatus("Motor ON");
        }

        else if (cmd == '0')
        {
          dutyCycle = 0;
          motorEnable = false;

          applyMotor();

          Serial.println("Motor OFF");
        }

        else if (cmd == '+')
        {
          dutyCycle += DUTY_STEP;

          if (dutyCycle > DUTY_MAX)
            dutyCycle = DUTY_MAX;
```

```
          motorEnable = true;

          applyMotor();

          tampilkanStatus("PWM NAIK");
        }

        else if (cmd == '-')
        {
          dutyCycle -= DUTY_STEP;

          if (dutyCycle < DUTY_MIN)
            dutyCycle = DUTY_MIN;

          motorEnable = (dutyCycle > 0);

          applyMotor();

          tampilkanStatus("PWM TURUN");
        }

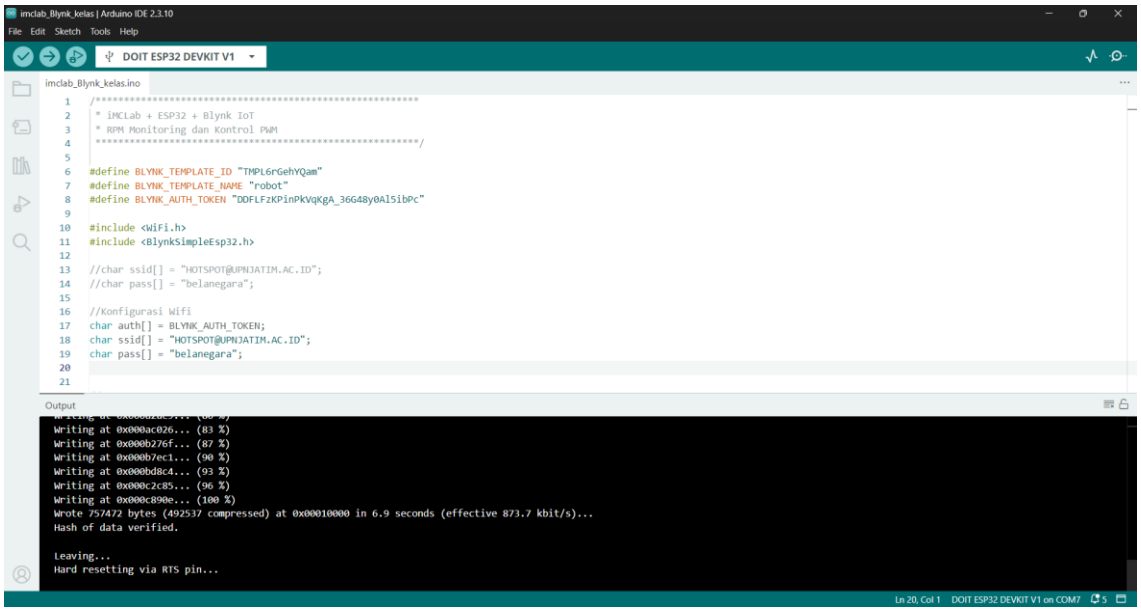
        else if (cmd == 'R' || cmd == 'r')
        {
          Serial.print("RPM = ");
          Serial.print(rpm, 1);
          Serial.print(" | PWM = ");
          Serial.println(dutyCycle);
        }
      }

      inputString = "";
    }
    else
    {
      inputString += c;
    }
  }
}
```

BAB II HASIL DAN PEMBAHASAN

2.1 Proses Upload Program ke ESP32

Sebelum upload dilakukan, library Blynk versi 1.4.0 beserta dependensinya dipasang melalui Library Manager, kemudian Template ID, Template Name, Auth Token, serta nama dan kata sandi WiFi disesuaikan dengan milik sendiri.

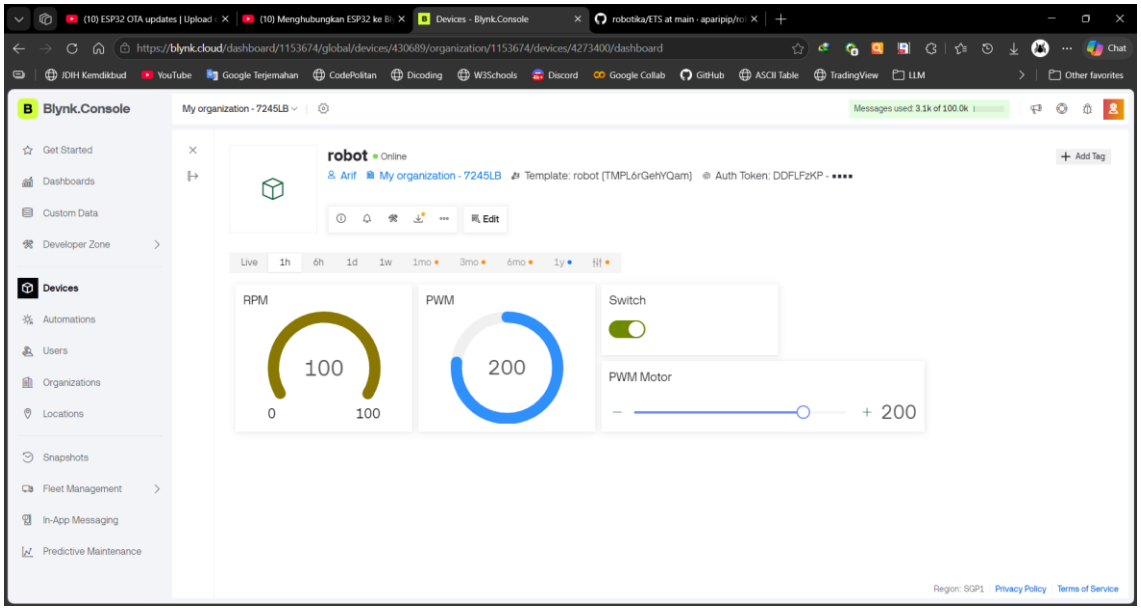


Gambar 1. Proses upload program Blynk ke board ESP32

Jendela Output menampilkan pemakaian memori sebesar 750.897 byte atau 57% dari kapasitas program, jauh lebih besar dibandingkan program pada praktikum sebelumnya karena bertambahnya pustaka WiFi dan Blynk. Proses penulisan flash berakhir dengan pesan “Hash of data verified” dan “Hard resetting via RTS pin”, yang menandakan upload berhasil dan board langsung melakukan restart.

2.2 Koneksi Perangkat ke Server Blynk

Setelah restart, ESP32 menyambung ke WiFi lalu mendaftarkan diri ke server Blynk Cloud region SGP1 menggunakan Auth Token.

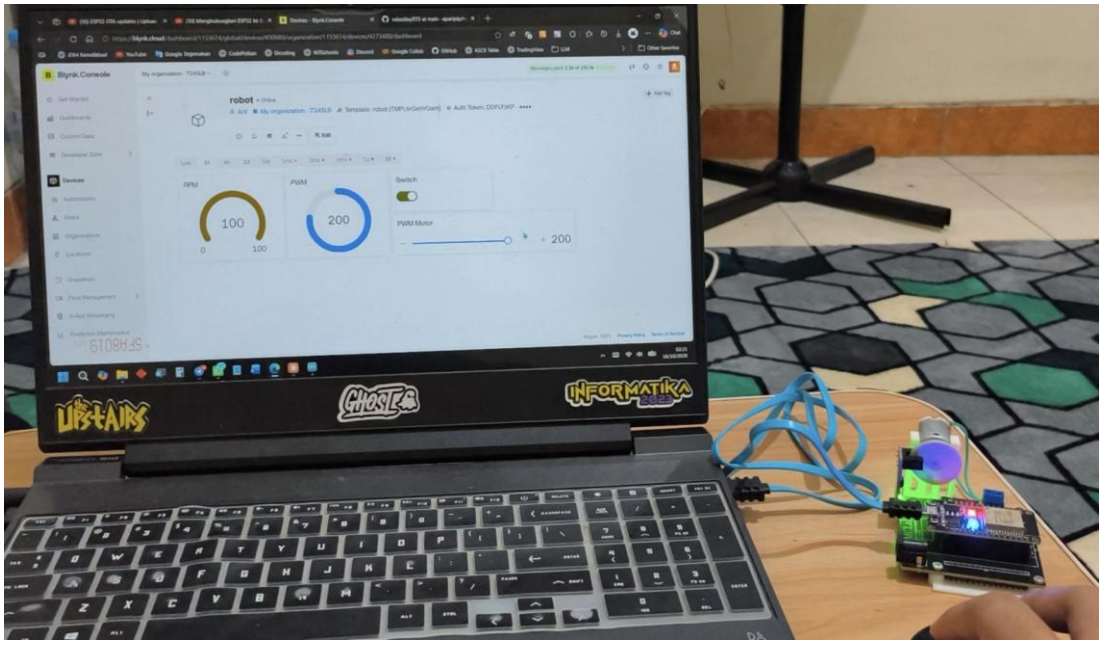


Gambar 2. Dashboard Blynk saat perangkat berstatus Online

Pada dashboard, nama device “robot” disertai penanda hijau bertuliskan Online, yang berarti sambungan ke server berhasil. Widget PWM dan slider PWM Motor menampilkan nilai 200, sedangkan Switch berada pada posisi aktif. Nilai ini berasal dari variabel dutyCycle yang bernilai awal 200 dan dikirimkan ke dashboard oleh fungsi kirimKeBlynk(), sehingga tampilan dashboard langsung mencerminkan kondisi perangkat tanpa perlu digerakkan terlebih dahulu.

2.3 Pengujian Tombol Switch (Motor ON)

Pengujian dilanjutkan dengan mengamati kondisi fisik perangkat ketika Switch pada dashboard berada di posisi ON.

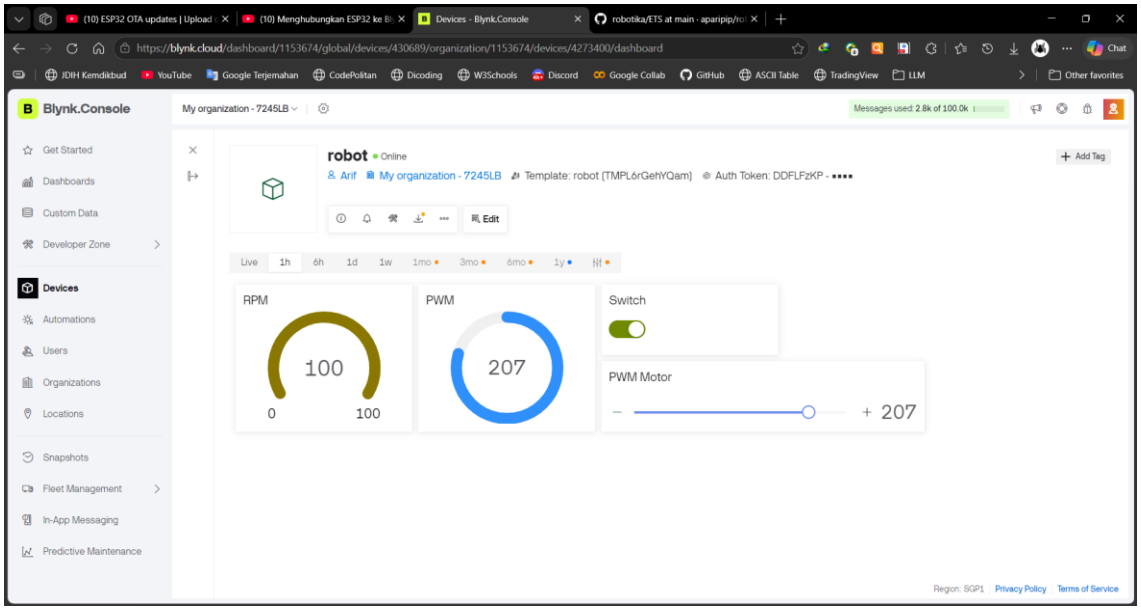


Gambar 3. Pengujian langsung dashboard Blynk dengan KIT iMCLab

Pada gambar tersebut terlihat dashboard Blynk di layar laptop menampilkan PWM 200 dengan Switch aktif, sementara KIT iMCLab di sebelah kanan menunjukkan piringan motor yang berputar hingga tampak kabur dan LED indikator biru yang menyala. Perintah yang dikirim dari browser melewati server Blynk Cloud terlebih dahulu sebelum diterima ESP32, namun jeda yang terjadi tidak terasa karena hanya dalam orde puluhan milidetik.

2.4 Menaikkan Kecepatan melalui Slider

Pengaturan kecepatan dilakukan dengan menggeser slider PWM Motor ke kanan.

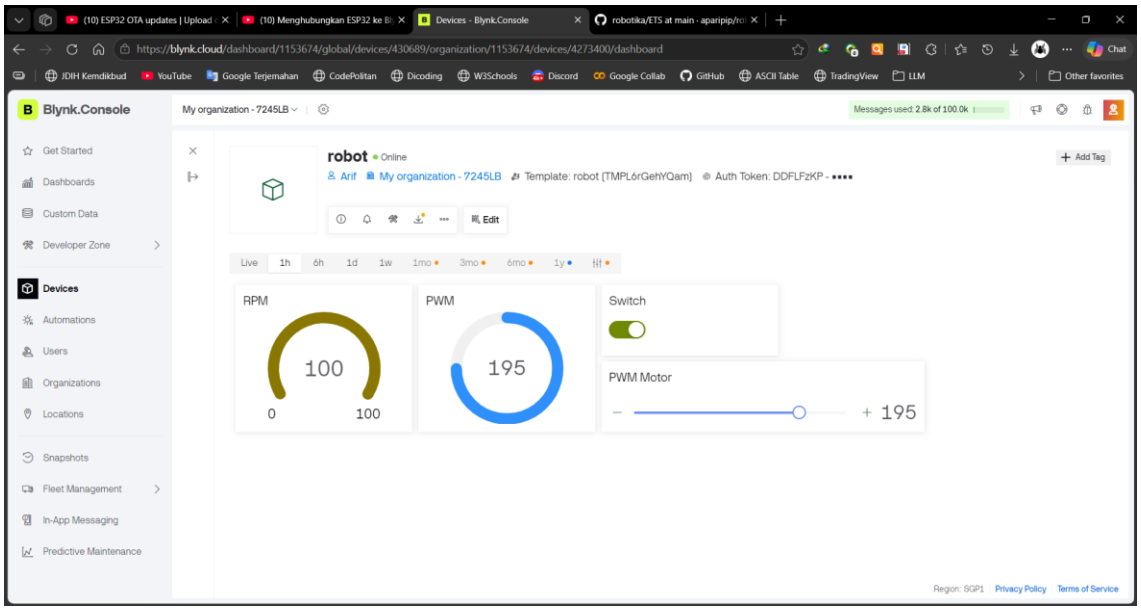


Gambar 4. Kecepatan dinaikkan melalui slider menjadi PWM 207

Slider digeser hingga menunjukkan nilai 207 dan widget PWM ikut berubah menjadi 207. Alurnya adalah slider mengirim nilai ke V2, fungsi BLYNK_WRITE(V2) membatasinya dengan constrain() pada rentang 0–255, lalu applyMotor() menuliskannya ke kanal PWM dan mengirim balik nilai tersebut ke V1. Berbeda dengan perintah ‘+’ yang hanya menambah 10 satuan setiap penekanan, slider memungkinkan nilai mana pun dipilih dalam satu gerakan.

2.5 Menurunkan Kecepatan melalui Slider

Pengujian kebalikannya dilakukan dengan menggeser slider ke kiri.

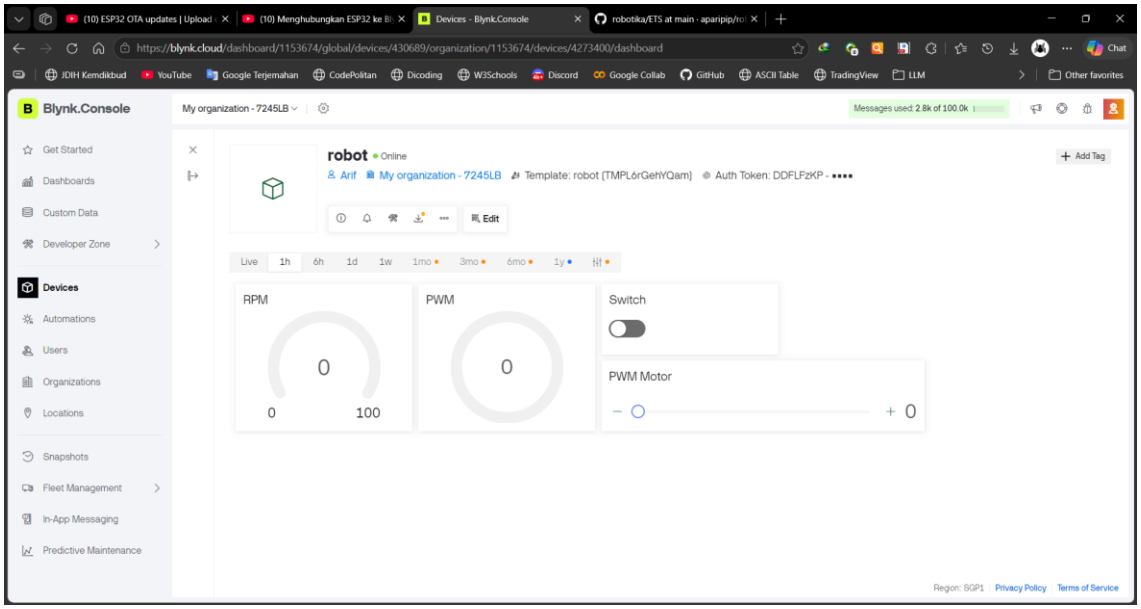


Gambar 5. Kecepatan diturunkan melalui slider menjadi PWM 195

Nilai duty cycle turun menjadi 195 dan widget PWM kembali menyesuaikan. Putaran motor melambat seiring berkurangnya daya rata-rata yang diterima. Karena nilai PWM dan posisi slider selalu disinkronkan oleh program, kedua widget tidak pernah menampilkan angka yang berbeda.

2.6 Mematikan Motor melalui Switch

Pengujian terakhir dilakukan dengan menekan Switch ke posisi OFF.



Gambar 6. Kondisi dashboard setelah Switch dimatikan

Seluruh widget kembali ke angka 0, yaitu RPM 0, PWM 0, dan slider 0. Hal ini sesuai dengan fungsi BLYNK_WRITE(V3) yang menetapkan dutyCycle = 0 ketika tombol dimatikan, kemudian applyMotor() memadamkan LED dan menghentikan motor. Nilai slider ikut kembali ke nol karena applyMotor() juga menuliskan duty cycle terbaru ke V2, sehingga tidak ada widget yang tertinggal pada nilai lama.

2.7 Pembacaan RPM dan Keterbatasan Skala Gauge

Fitur yang tidak dimiliki dua praktikum sebelumnya adalah pemantauan RPM. Nilai RPM dihitung pada fungsi hitungRPM() dengan membagi jumlah pulsa encoder dengan jumlah lubang piringan (HOLES = 2), lalu dikalikan 60 untuk memperoleh putaran per menit. Pembacaan pulsa dilakukan melalui

interrupt pada pin 13 dengan penapis debounce 500 mikrodetik agar derau tidak terhitung sebagai pulsa.

Pada pengujian ini gauge RPM selalu menunjukkan angka 100 ketika motor berputar, padahal batas maksimum datastream V0 juga diatur 100. Artinya jarum gauge berada pada posisi mentok dan nilai RPM sesungguhnya kemungkinan besar lebih tinggi dari angka yang ditampilkan, karena Blynk memotong nilai yang melebihi batas datastream. Nilai aslinya dapat diperiksa melalui Serial Monitor dengan perintah 'R', yang menampilkan RPM tanpa pembatasan skala. Temuan ini menunjukkan pentingnya menyesuaikan rentang datastream dengan rentang nilai yang benar-benar mungkin terjadi pada sistem.

2.8 Rangkuman Hasil Pengujian

No	Aksi pada Dashboard	Hasil Pengamatan
1	Perangkat dinyalakan	Device berstatus Online, PWM dan slider menampilkan 200
2	Switch ON	Motor berputar, LED biru menyala, PWM = 200
3	Slider digeser ke kanan	PWM naik menjadi 207, putaran motor bertambah cepat
4	Slider digeser ke kiri	PWM turun menjadi 195, putaran motor melambat
5	Switch OFF	RPM, PWM, dan slider menjadi 0, motor berhenti, LED padam
6	Gauge RPM	Tertahan di 100 karena batas maksimum datastream masih 100

Seluruh aksi pada dashboard memberikan tanggapan yang sesuai pada perangkat keras, dan nilai antar widget selalu konsisten karena program melakukan sinkronisasi dua arah setiap kali terjadi perubahan.

BAB III PENUTUP

3.1 Kesimpulan

Berdasarkan hasil pengujian, dapat disimpulkan beberapa hal sebagai berikut:

- Board ESP32 berhasil terhubung ke jaringan WiFi dan server Blynk Cloud menggunakan Template ID dan Auth Token, ditandai dengan status Online pada dashboard.
- Motor DC dapat dinyalakan dan dimatikan melalui widget Switch (V3), serta kecepatannya dapat diatur melalui widget Slider (V2) pada rentang duty cycle 0–255.
- Nilai PWM pada dashboard selalu sinkron dengan kondisi perangkat karena fungsi applyMotor() dan kirimKeBlynk() menuliskan kembali nilai terbaru ke virtual pin.
- Pengujian pada PWM 200, 207, dan 195 menunjukkan perubahan kecepatan putaran yang sesuai, sedangkan PWM 0 menghentikan motor dan memadamkan LED indikator.
- Pembacaan RPM melalui sensor encoder sudah berjalan, namun tampilannya tertahan pada angka 100 karena batas maksimum datastream V0 belum disesuaikan.
- Dibandingkan dua praktikum sebelumnya, pengendalian berbasis Blynk tidak lagi memerlukan kabel data ke komputer sehingga motor dapat dikendalikan dari mana saja selama perangkat terhubung internet.